# 1. RPC(Giới thiệu qua về RPC):

- Remote Procedure Call (RPC) và REST là hai kiểu kiến trúc trong thiết kế API. API là cơ chế cho phép 2 thành phần phần mềm giao tiếp với nhau bằng một tập hợp các định nghĩa và giao thức. Các nhà phát triển phần mềm sử dụng các thành phần được phát triển trước đó hoặc của bên thứ ba để thực hiện các hàm, vì vậy họ không phải viết mọi thứ từ đầu. RPC API cho phép nhà phát triển gọi các hàm từ xa trong các máy chủ bên ngoài như thể những chức năng đó nằm trong phần mềm của họ. Ví dụ: Bạn có thể thêm chức năng trò chuyện vào ứng dụng bằng cách gọi từ xa các hàm nhắn tin trên một ứng dụng trò chuyện khác. Ngược lại, API REST cho phép bạn thực hiện các hoạt động dữ liệu cụ thể trên một máy chủ từ xa. Ví dụ: Ứng dụng của bạn có thể chèn hoặc sửa đổi dữ liệu nhân viên trên máy chủ từ xa bằng API REST.

| | RPC | REST |
|---|---|---|
| Đó là gì? | Một hệ thống cho phép máy khách từ xa gọi thủ tục trên máy chủ dưới dạng cục bộ. | Một tập hợp các quy tắc định hình quá trình trao đổi dữ liệu có cấu trúc giữa một máy khách và một máy chủ. |
| Trường hợp sử dụng | Thực hiện các hành động trên máy chủ từ xa. | Các thao tác tạo, đọc, cập nhật và xóa (CRUD) trên các đối tượng từ xa. |
| Phù hợp nhất | Khi yêu cầu tính toán phức tạp hoặc kích hoạt quy trình từ xa trên máy chủ. | Khi dữ liệu máy chủ và cấu trúc dữ liệu cần được trình bày thống nhất. |
| Có trạng thái | Phi trạng thái hoặc có trạng thái. | Phi trạng thái. |

| | | |
|---|---|---|
| Định dạng truyền dữ liệu | Trong một cấu trúc nhất quán được xác định bởi máy chủ và được thực thi trên máy khách. | Trong một cấu trúc được xác định độc lập bởi máy chủ. Nhiều định dạng khác nhau có thể được truyền trong cùng một API. |

# 2. Các loại RPC:

## 2.1 Theo cách chờ kết quả

- **RPC đồng bộ — Synchronous RPC:** luồng gọi chờ kết quả trước khi tiếp tục thực hiện công việc.
- **RPC bất đồng bộ — Asynchronous RPC:** chương trình khởi tạo lời gọi rồi tiếp nhận kết quả qua callback, future hoặc cơ chế tương đương, giúp luồng hiện tại tiếp tục xử lý công việc khác. API cụ thể phụ thuộc ngôn ngữ và thư viện.
- Ví dụ, ứng dụng chat có thể lấy thông tin phòng theo cách bất đồng bộ để giao diện vẫn phản hồi khi người dùng thao tác.

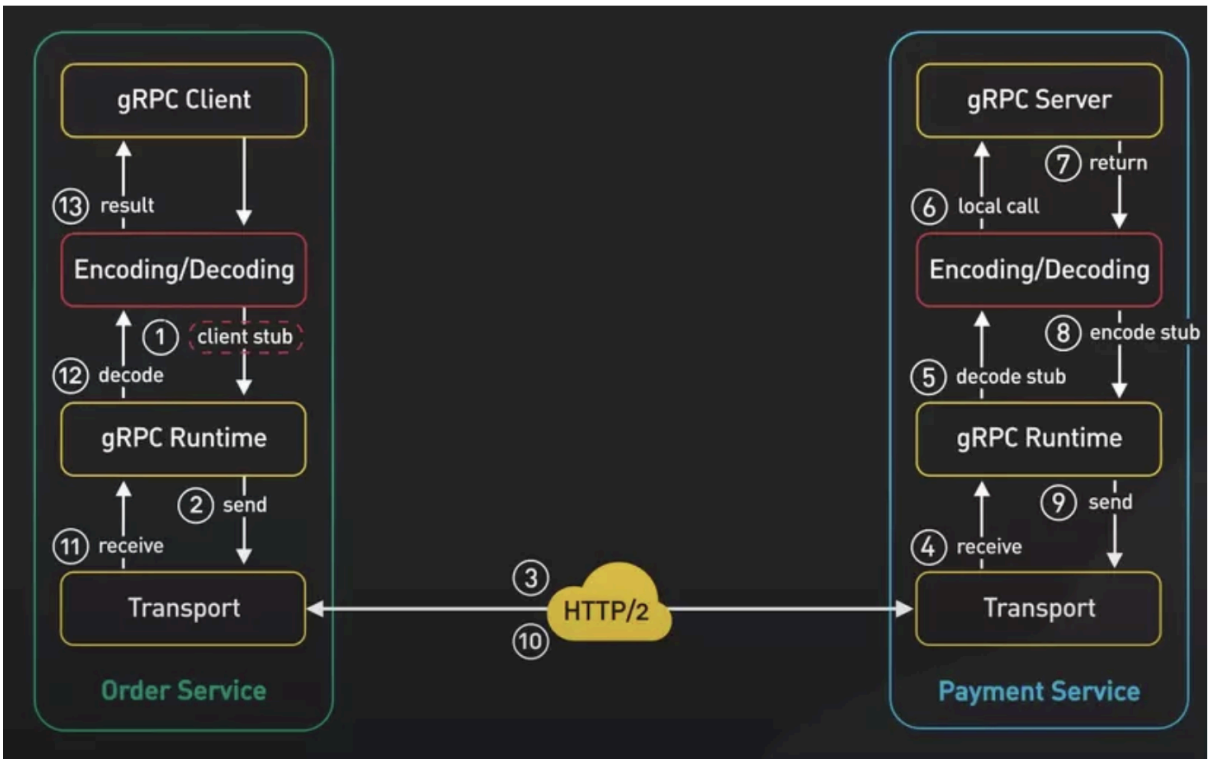
## 2.2 Theo công nghệ triển khai

| Công nghệ | Đặc điểm |
|---|---|
| **XML-RPC** | Biểu diễn lời gọi và kết quả bằng XML, sử dụng HTTP POST. |
| **JSON-RPC** | Quy định cấu trúc lời gọi và phản hồi bằng JSON; không phụ thuộc một cơ chế vận chuyển duy nhất. |
| **Java RMI** | Cho phép gọi phương thức trên đối tượng Java từ xa. |
| **gRPC** | Framework RPC, thường sử dụng Protocol Buffers và truyền dữ liệu qua HTTP/2. |

- Các công nghệ này cùng phục vụ việc gọi chức năng từ xa nhưng khác nhau về định dạng dữ liệu, hệ sinh thái và cách triển khai.
- Một API sử dụng POST và JSON chưa đủ để được gọi là JSON-RPC. JSON-RPC 2.0 quy định các trường như `jsonrpc`, `method`, `params`, `id` và cấu trúc phản hồi tương ứng.

# 3. gRPC, Protocol Buffers và HTTP/2

gRPC là một framework RPC (Remote Procedure Call) mã nguồn mở, được sử dụng để xây dựng các API có khả năng mở rộng và tốc độ cao. Nó giúp các ứng dụng client và server giao tiếp dễ dàng và phát triển các hệ thống kết nối. gRPC sử dụng HTTP/2, Protocol Buffers (Protobuf) làm chủ đạo và các công nghệ khác để đảm bảo bảo mật, hiệu suất và khả năng mở rộng tối ưu cho API.




**a) Các khái niệm cơ bản trong gRPC:**

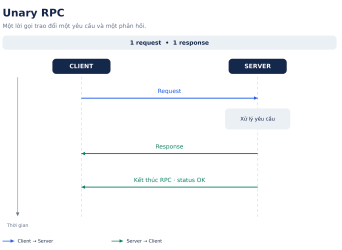
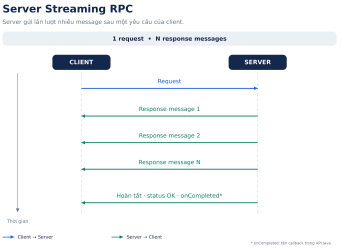
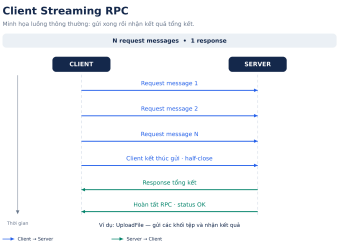
Protocol Buffers (Protobuf): Là một serialization/deserialization protocol của Google, giúp định nghĩa các services. Protobuf giúp gRPC tiết kiệm tài nguyên CPU hơn so với JSON/XML nhờ vào việc chuyển đổi dữ liệu thành định dạng nhị phân.

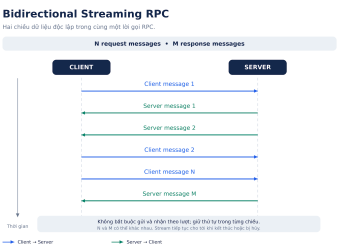
- Streaming: gRPC hỗ trợ ba kiểu streaming:
  1. Server-streaming: Client gửi một yêu cầu đơn lẻ và nhận về một stream dữ liệu từ server. Client đọc dữ liệu streaming trả về cho đến khi không còn message nào nữa. gRPC đảm bảo thứ tự message trong một RPC call riêng lẻ.
  2. Client-streaming: Client gửi một stream dữ liệu tới server (provided stream) một cách tuần tự, server sẽ xử lý yêu cầu và trả về cho client một response duy nhất.
  3. Bidirectional-streaming: Cả client và server đều gửi và nhận các message (read-write stream). Cả hai luồng hoạt động độc lập với nhau; Vì thế mà chúng

có thể truyền message theo bất kỳ trình tự nào. tuy nhiên thì trình tự tin nhắn trong mỗi luồng được giữ nguyên.

- HTTP/2: gRPC sử dụng HTTP/2, mang lại khả năng truyền tải dữ liệu hiệu quả hơn, hỗ trợ multiplexing (đa luồng), streaming hai chiều, kiểm soát luồng, nén header và khả năng xử lý đồng thời.
- Channels: Là các kết nối giữa client và server, hỗ trợ nhiều luồng đồng thời qua một kết nối TCP duy nhất.

**2. Nắm rõ 4 kiểu RPC — đây là trọng tâm**

| Kiểu RPC | Cách hoạt động | Ảnh minh họa |
|---|---|---|
| **Unary** | Client gửi 1 yêu cầu, server trả 1 phản hồi |  |
| **Server streaming** | Client gửi 1 yêu cầu, server trả nhiều thông điệp<br>Client gửi một yêu cầu đơn lẻ và nhận về một stream dữ liệu từ server. Client đọc dữ liệu streaming trả về cho đến khi không còn message nào nữa. gRPC đảm bảo thứ tự message trong một RPC call riêng lẻ. |  |
| **Client streaming** | Client gửi nhiều thông điệp, server trả 1 kết quả cuối<br>Client gửi một stream dữ liệu tới server (provided stream) một cách tuần tự, server sẽ xử lý yêu cầu và trả về cho client một response duy nhất. |  |

| | | |
|---|---|---|
| **Bidirectional streaming** | Hai bên liên tục gửi và nhận, độc lập với nhau<br><br>Cả client và server đều gửi và nhận các message (read-write stream). Cả hai luồng hoạt động độc lập với nhau; Vì thế mà chúng có thể truyền message theo bất kỳ trình tự nào. tuy nhiên thì trình tự tin nhắn trong mỗi luồng được giữ nguyên. |  |

Ảnh BTL

# Protocol Buffer:

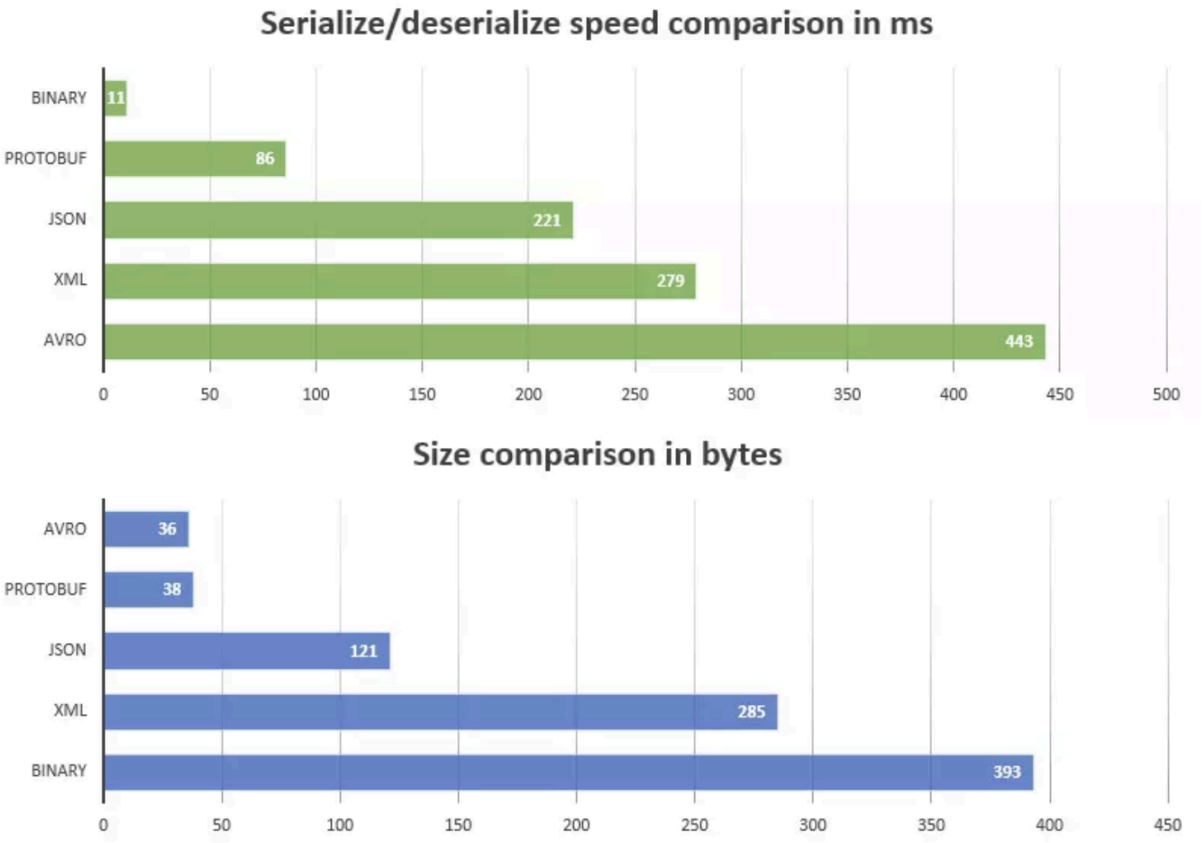
**Protocol Buffers (Protobuf) là cách mô tả dữ liệu có cấu trúc và chuyển dữ liệu đó thành dạng nhị phân để lưu trữ hoặc truyền qua mạng.** Nó phục vụ mục đích tương tự JSON, nhưng yêu cầu khai báo cấu trúc dữ liệu trước trong file `.proto`.

Ví dụ: với JSON {"id":21,"name":"Minh"} có 23 bytes

Protobuf tương ứng có thể biểu diễn bằng **8 byte**, viết dưới dạng hexadecimal:

08 15 12 04 4D 69 6E 68

Nó là một giải pháp hiệu quả để tạo dữ liệu có cấu trúc cho các ứng dụng khác nhau như ứng dụng web, hệ thống gRPC,... và đặc biệt hiệu dụng trong microservices sử dụng gRPC.




Ưu điểm:

- schema rõ ràng;
- kích thước dữ liệu thường nhỏ;
- serialization/deserialization hiệu quả;
- hỗ trợ nhiều ngôn ngữ.

## HTTP2

## Tài liệu tham khảo:

[1] https://grpc.io/docs/what-is-grpc/introduction/
[2] https://viblo.asia/p/grpc-no-la-gi-va-co-nen-su-dung-hay-khong-gDVK2mAj5Lj
[3] https://aws.amazon.com/vi/compare/the-difference-between-rpc-and-rest/
[4]